

Develop a program to implement the Naive Bayesian classifier considering Olivetti Face Data set for training. Compute the accuracy of the classifier, considering a few test data sets.

```
import numpy as np
from sklearn.datasets import fetch_olivetti_faces
from sklearn.naive_bayes import GaussianNB
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, classification_report
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt

# Load the Olivetti Faces dataset
olivetti = fetch_olivetti_faces()
X = olivetti.data #Stores the face images as data points (features).
y = olivetti.target # Stores the corresponding labels (person's identity)
                    #for each face image.

print(f"Dataset loaded: {X.shape[0]} samples, {X.shape[1]} features,
      {len(np.unique(y))} classes")

# Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
                                                    random_state=42, stratify=y)

print(f"Training set size: {X_train.shape[0]}")
print(f"Test set size: {X_test.shape[0]}")

# Apply PCA for dimensionality reduction
n_components = 100
pca = PCA(n_components=n_components, whiten=True, random_state=42)
X_train_pca = pca.fit_transform(X_train)
X_test_pca = pca.transform(X_test)

print(f"Reduced dimensions after PCA: {X_train_pca.shape[1]}")

# Initialize and train the Gaussian Naive Bayes classifier
gnb = GaussianNB()
gnb.fit(X_train_pca, y_train)

# Make predictions on the test set
y_pred = gnb.predict(X_test_pca)

# Calculate accuracy
accuracy = accuracy_score(y_test, y_pred)
print(f"\nAccuracy: {accuracy:.2%}")
```

```

# Print classification report
print("\nClassification Report:")
print(classification_report(y_test, y_pred))

# Visualize some test results
plt.figure(figsize=(10, 8))
for i in range(12):
    plt.subplot(3, 4, i+1)
    plt.imshow(X_test[i].reshape(64, 64), cmap='gray')
    plt.title(f"True: {y_test[i]}\nPred: {y_pred[i]}")
    plt.axis('off')
plt.tight_layout()
plt.show()

```

Output:

Dataset loaded: 400 samples, 4096 features, 40 classes
 Training set size: 320
 Test set size: 80
 Reduced dimensions after PCA: 100

Accuracy: 88.75%

Classification Report:				
	precision	recall	f1-score	support
0	0.67	1.00	0.80	2
1	1.00	1.00	1.00	2
2	1.00	1.00	1.00	2
3	0.67	1.00	0.80	2
4	0.00	0.00	0.00	2
5	1.00	1.00	1.00	2
6	1.00	0.50	0.67	2
7	0.67	1.00	0.80	2
8	1.00	1.00	1.00	2
9	1.00	0.50	0.67	2
10	1.00	1.00	1.00	2
11	1.00	1.00	1.00	2
12	0.33	0.50	0.40	2
13	1.00	1.00	1.00	2
14	1.00	1.00	1.00	2
15	1.00	1.00	1.00	2
16	1.00	1.00	1.00	2
17	1.00	0.50	0.67	2
18	1.00	1.00	1.00	2
19	1.00	1.00	1.00	2
20	1.00	1.00	1.00	2
21	1.00	1.00	1.00	2
22	1.00	0.50	0.67	2
23	1.00	1.00	1.00	2
24	1.00	1.00	1.00	2
25	0.50	1.00	0.67	2
26	1.00	1.00	1.00	2
27	1.00	1.00	1.00	2
28	1.00	1.00	1.00	2
29	1.00	0.50	0.67	2
30	1.00	0.50	0.67	2
31	1.00	1.00	1.00	2
32	1.00	1.00	1.00	2
33	1.00	1.00	1.00	2
34	1.00	1.00	1.00	2
35	0.67	1.00	0.80	2
36	1.00	1.00	1.00	2
37	1.00	1.00	1.00	2
38	1.00	1.00	1.00	2
39	0.67	1.00	0.80	2
accuracy			0.89	80
macro avg	0.90	0.89	0.88	80
weighted avg	0.90	0.89	0.88	80

